

YOLOv8n-seg-sphflame 用户文档

1. 环境搭建

1.1 开发环境

- IDE：PyCharm Community Edition 2025.1.3.1（或其他支持 Python 的编辑器）
- Python 环境：推荐使用 Anaconda 创建虚拟环境，命名为 `YOLO`
- Python 版本：3.8.0

1.2 依赖库

在虚拟环境中安装以下核心依赖（版本为推荐版本，兼容性经过验证）：

库名	版本	说明
labelme	3.16.7	图像多边形标注工具
numpy	1.24.4	数值计算
matplotlib	3.7.5	绘图与可视化
opencv-python	4.12.0.88	图像处理
pandas	2.0.3	数据处理
torch	1.13.0+cu117	PyTorch 深度学习框架（GPU）
torchaudio	0.13.0+cu117	音频处理（PyTorch 生态）
torchvision	0.14.0+cu117	计算机视觉工具包
ultralytics	8.3.203	YOLOv8 训练与推理框架

注意：

若使用 CPU 训练，可将 `torch`、`torchaudio`、`torchvision` 的 CUDA 版本替换为 CPU 版本。

安装示例（请先激活虚拟环境 `conda activate YOLO`）：

```
1 pip install labelme==3.16.7 numpy==1.24.4 matplotlib==3.7.5 opencv-python==4.12.0.88
   pandas==2.0.3
2 pip install torch==1.13.0+cu117 torchaudio==0.13.0+cu117 torchvision==0.14.0+cu117 -f
   https://download.pytorch.org/whl/torch_stable.html
3 pip install ultralytics==8.3.203
```

⚠ Caution

部分组件安装时会自动安装依赖组件的更高版本，导致依赖该依赖组件的部分组件不可用。总之，在全部所需组件安装完毕后，激活虚拟环境后使用 `conda list` 命令查看各组件及其对应版本。

2. 数据准备

若直接使用预训练模型进行预测，可跳过本章。

2.1 概述

模型专注于球形火焰边缘的像素级分割。为了获得高精度的分割效果，建议针对实际场景制作专用数据集。制作流程为：

1. 使用 **labelme** 对火焰纹影图像进行多边形标注，生成 JSON 文件。
2. 通过转换脚本将 JSON 格式的数据集转换为 YOLO 格式，并自动划分训练集与验证集。

数据集越丰富、标注越精确，模型的实际识别能力越强。

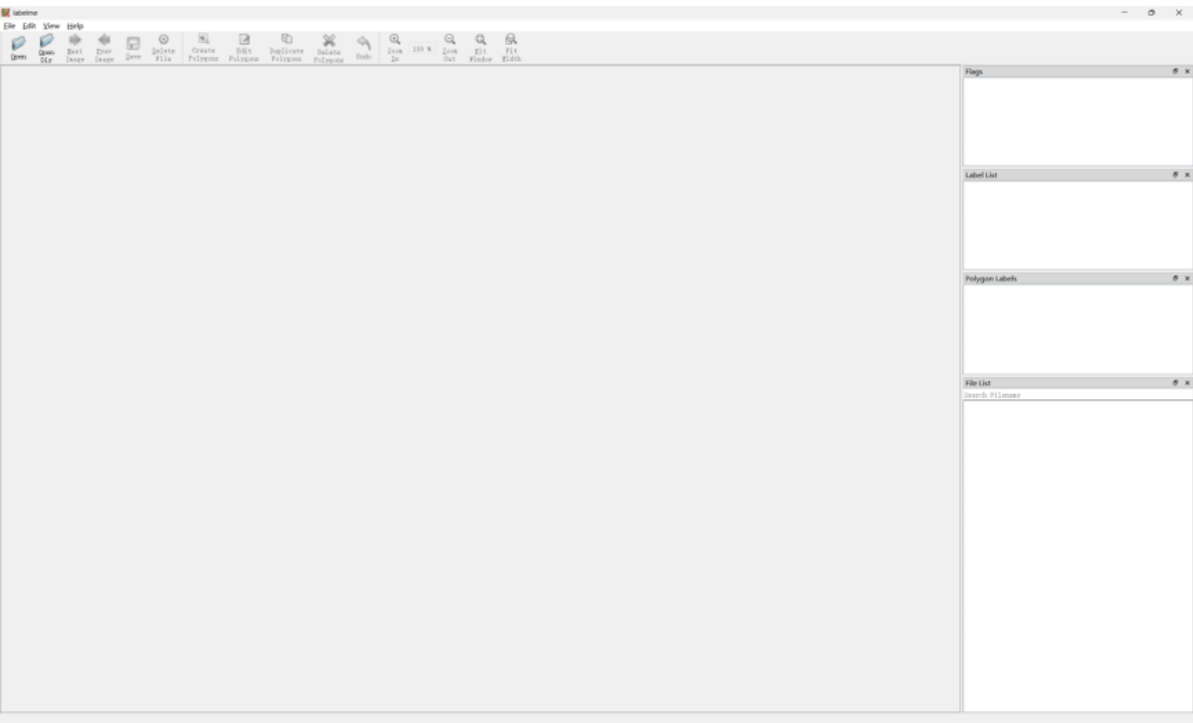
2.2 数据标注

2.2.1 启动 labelme

打开 Anaconda 命令提示符，依次执行：

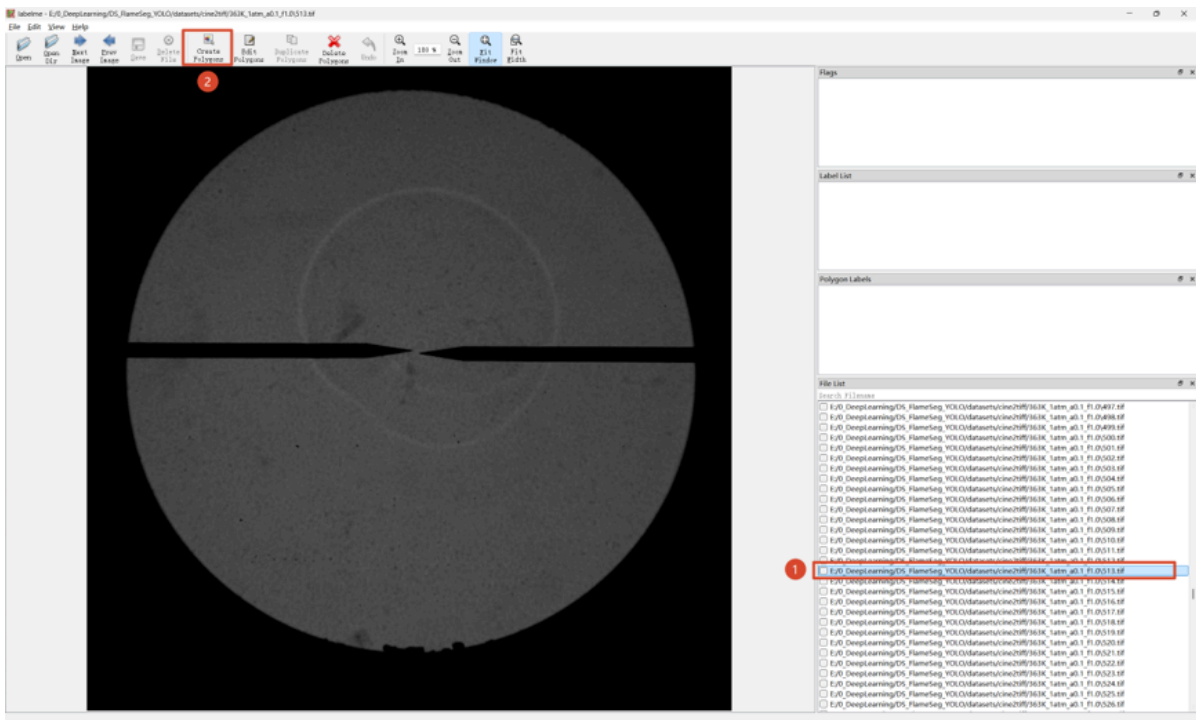
```
1 conda activate YOLO
2 labelme
```

成功启动后，界面如下图所示。



2.2.2 加载图片

点击 **Open Dir**，选择存放原始火焰图片的文件夹（笔者使用的文件夹为 `./dataset/cine2tiff/`），左侧文件列表会显示所有图片，单击即可开始标注。



2.2.3 标注对象与方式

本模型训练需要标注三类对象：

对象	标签名	标注方式	用途
圆形视窗	window	矩形框	提供物理尺度参考，消除像素比例尺偏差
火焰边缘	flame	多边形	用于计算火焰半径
高频火花	spark	多边形	排除火花对火焰识别的干扰

具体操作：

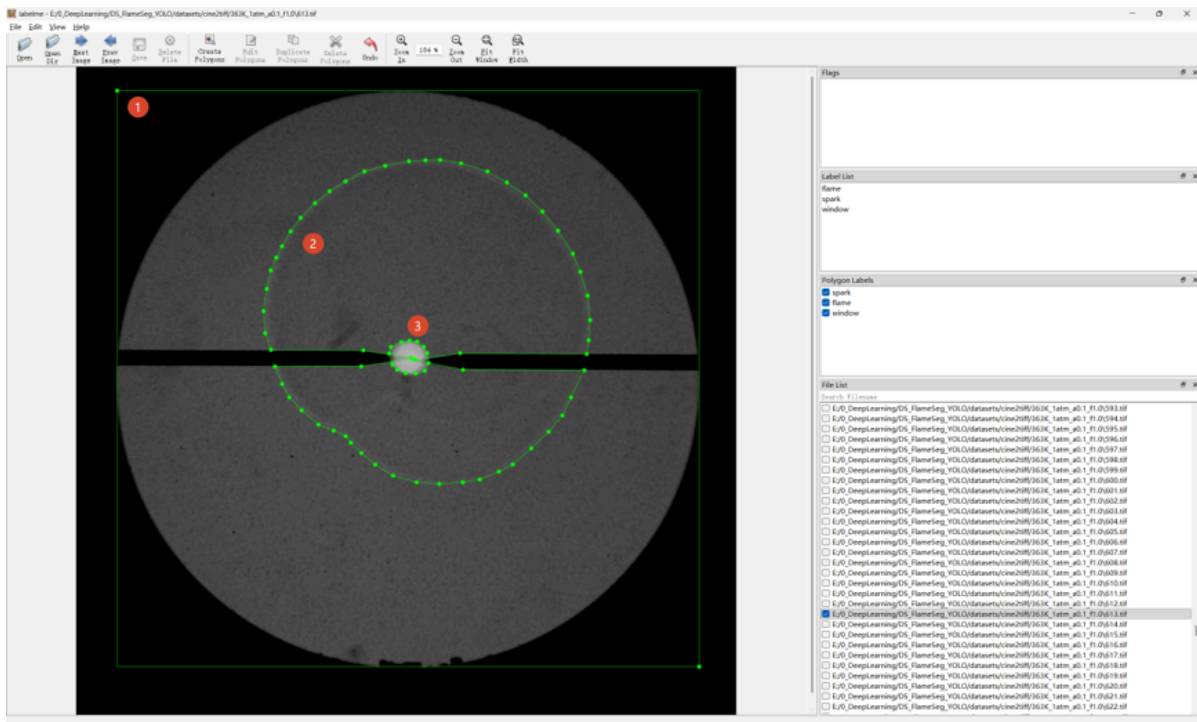
- **圆形视窗：** Edit → Create Rectangle（快捷键 **Ctrl+R**），框选出视窗区域。
- **火焰边缘：** Edit → Create Polygons（快捷键 **Ctrl+N**），沿火焰外边缘逐点描绘（当被电极覆盖时，同样需要绕过电极轮廓进行标注），闭合多边形。标注点不可压在火焰锋面上。
- **高频火花：** 同样使用多边形标注（**Ctrl+N**），描绘火花区域。火花亮度高、可能覆盖电极时则不必刻意避让电极。

2.2.4 标注顺序与覆盖规则

labelme 支持同一图像内多个多边形重叠，生成语义分割掩膜时 **每个像素仅保留最后一次标注的类别**。因此，建议标注顺序为：

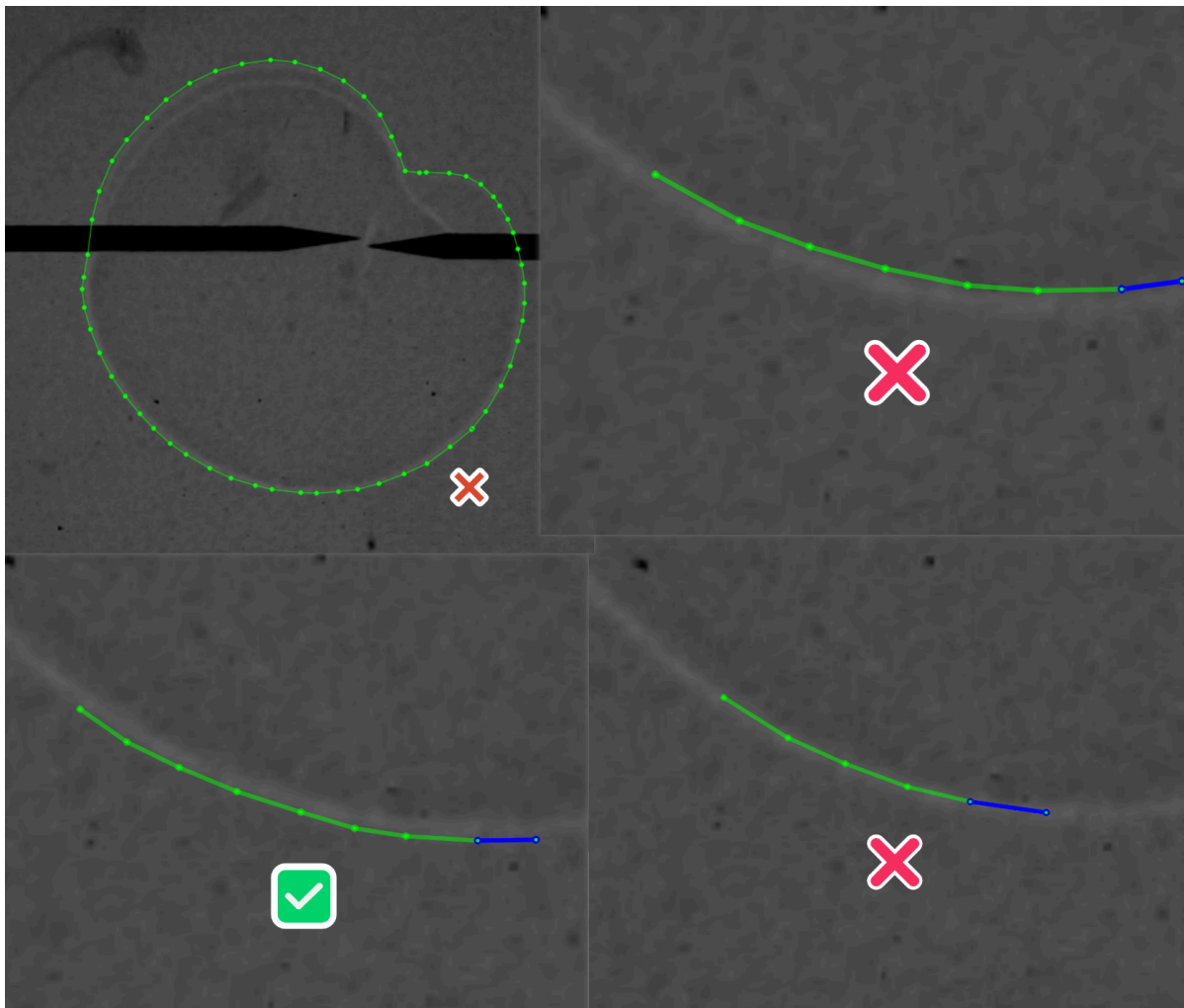
1. 视窗（**window**）
2. 火焰（**flame**）
3. 火花（**spark**）

即 **内层类别覆盖外层类别**，确保最终掩膜中火花区域的像素不会错误地标记为火焰或视窗。



标注正误示范：

- ✗ 未绕过电极进行标注
- ✗ 标注火焰时标注点在火焰锋面内侧（未将火焰锋面置于标注轮廓内）
- ✗ 标注火焰时标注点压在火焰边缘上（只包含了部分火焰锋面）
- ✓ 标注点在火焰锋面外侧（包含了全部火焰信息）



2.2.5 保存标注

每完成一张图片的标注，按下 `Ctrl+S` 保存，或通过 `File → Save Automatically` 开启自动保存。标注文件与图片同名，扩展名为 `.json`。可按 `A / D` 键快速切换到上一张或下一张图片。

2.2.6 整理文件

通过手动或自动方式将已标注图片及对应的 JSON 文件放入对应文件夹中。

手动方式： 将所有标注好的原始图片（`.jpg` 等）放入 `.\dataset\segment\img` 文件夹，对应的 JSON 文件放入 `.\dataset\segment\json` 文件夹。目录结构示例：

自动方式： 将 `move.bat` 脚本放置在标注的文件夹中（笔者的文件夹为 `.\dataset\cine2tiff`），运行脚本即可将该文件夹中已标注的文件及图片存放至该文件夹下的 `img` 及 `json` 文件夹中，再将这两个文件夹移动至 `.\dataset\segment` 下。

最终，进入数据类型转换与数据集划分环节时的文件结构如下：

```
1  dataset/
2    └─ segment/
3        └─ img/      # 原始图片
4            └─ 000.jpg
5            └─ 001.jpg
6            └─ ...
7        └─ json/     # labelme 标注文件
8            └─ 000.json
9            └─ 001.json
10           └─ ...
```

2.3 数据类型转换与数据集划分

使用提供的 `Json2Yolo.py` 完成两项任务：

1. 将 labelme 的 JSON 标注转换为 YOLO 分割格式（每张图片对应一个 `.txt` 文件，存放多边形归一化坐标及类别 ID）。
2. 将数据集随机划分为训练集（`train`）和验证集（`val`）。

脚本的主函数中可设置划分比例，默认 `训练集:验证集 = 9:1`。如需调整，修改代码中对应变量，例如：

```
1  train_ratio = 0.8 # 训练集占 80%，验证集 20%
```

运行脚本后，将在 `.\dataset\segment\seg` 目录下生成如下结构：

```
1  dataset/
2    └─ segment/
3        └─ seg/      # 转换并划分后的 YOLO 数据集
4            └─ images/
5                └─ train/  # 训练集图片
6                    └─ 000.jpg
7                    └─ ...
8                └─ val/    # 验证集图片
9                    └─ 002.jpg
10               └─ ...
11           └─ labels/
```

```
12      |— train/      # 训练集标签 (txt 格式)
13      |   |— 000.txt
14      |   |— ...
15      |— val/        # 验证集标签
16      |   |— 002.txt
17      |   |— ...
```

重要：验证集仅用于监控训练过程中的模型性能，切勿将最终待预测的测试图片混入训练集或验证集中。

3. 模型训练

若直接使用预训练权重进行预测，可跳过本章。

3.1 训练脚本与配置

训练入口为 `trainXXXX.py`（`XXXX` 对应更新日期），核心函数

`train_flame_segmentation_optimized` 提供了多档训练配置，可根据硬件资源自动降级或手动选择。

主要可调参数（以中档配置为例）：

- **输入尺寸**（`imgsz`）：768 像素
- **批次大小**（`batch`）：8
- **训练轮数**（`epochs`）：200
- **初始学习率**（`lr0`）：0.002
- **预训练权重**：`yolov8n-seg.pt`（基础权重）或 `YOLOv8n-seg-sphflame-v1.0.pt`（层流小组专用权重）
- **优化器**：`AdamW`
- **学习率策略**：余弦退火（`cos_lr=True`）

各配置位于脚本中的 `configs` 列表，从高资源需求到低资源需求依次排列，训练时若因显存不足（out of memory）报错，会自动尝试下一档配置，直至成功或全部失败。

3.2 自定义训练类别

模型默认训练 **单类**（仅 `flame`），脚本中的 `verify_annotations` 函数会自动过滤掉其他类别的标注（如 `window`、`spark`）。如需训练多类分割模型，请修改以下两处：

1. **数据配置文件**：`datasets/segment/seg/segment.yaml` 中的 `nc` 和 `names` 字段。
2. **`train.py` 中的验证函数**：移除或修改 `verify_annotations` 的过滤逻辑。

3.3 运行训练

确保数据路径与脚本内配置一致，然后在虚拟环境中执行：

```
1 conda activate YOLO
2 python train.py
```

训练日志、权重文件及可视化结果将保存在 `flame_segmentation/` 项目目录下。

4. 模型预测

预测脚本 `predict_lfs_calculator.py` 集成了从语义分割、火焰边缘提取、半径拟合到层流火焰速度计算的完整流程。支持多种输入类型（图片序列、视频）和视窗标定方式，最终输出火焰传播速度、马克斯坦长度等关键燃烧参数。

4.1 脚本功能概述

- **语义分割**：选择 `YOLOv8n-seg` 基础模型或经过微调的 `YOLOv8n-seg-sphflame` 模型对输入帧进行推理，提取火焰区域掩膜。
- **边缘与轮廓处理**：从掩膜中提取火焰轮廓，通过可调节比例的点选择策略进行圆形拟合，得到火焰半径（像素单位）。
- **视窗标定**：支持三种方式确定像素到毫米的转换系数：
 - `yolo` —— 使用模型检测圆形视窗（`window` 类），结合霍夫圆检测计算比例尺；
 - `traditional` —— 使用传统形态学图像处理方法（高斯模糊、Canny、霍夫圆）检测视窗；
 - `manual` —— 手动输入固定的像素-毫米转换系数。
- **数据清洗与平滑**：采用移动中位数离群值检测、插值填充、Loess 平滑及卡尔曼滤波（可选），保证半径和速度曲线的可靠性。
- **层流火焰速度计算**：基于三种经典外推模型（线性、Frankel-Chen、Kelly-Law）计算无拉伸层流火焰速度 `Sb0` 和马克斯坦长度 `Lb`。
- **可视化与结果保存**：生成四象限诊断图（`rb-t`、`Sb-rb`、`Sb-t`、`Sb-α`），导出 CSV 结果文件及拟合曲线数据。

4.2 准备工作

- **模型权重**：将训练好的 `.pt` 文件（如 `YOLOv8n-seg-sphflame-v1.0.pt` 或您的 `best.pt`）放置于项目根目录或 `.\weights\` 文件夹内。
- **输入数据**：支持 TIFF/JPEG/PNG 图片序列（放在同一文件夹）或 MP4/AVI 等视频文件。请将待预测数据准备好，并记录其帧率（对于图片序列，须手动指定帧率 `--fps`）。

4.3 参数配置

脚本提供了两种设置参数的方式，任选其一：

4.3.1 方式一：直接修改代码中的默认值（推荐快速测试）

在 `predict_lfs_calculator.py` 中找到 `if __name__ == "__main__":` 下方的参数区域（约 1900 行），按需修改以下核心变量：

```
1  # ===== 可修改的参数区域 =====
2  # 1. 模型权重文件路径
3  MODEL_PATH = "weights/YOLOv8n-seg-sphflame-v1.0.pt" # 权重路径
4
5  # 2. 输入路径（视频文件或TIFF图片文件夹）
6  INPUT_PATH = "./datasets/cine2tiff/test" # 输入图片文件夹或视频文件
7
```



```

8      # 3. 输出目录
9      OUTPUT_DIR = "./integrated_results"          # 输出目录
10
11     # 4. 火焰半径范围 (单位: mm)
12     MIN_RADIUS_MM = 8                            # 有效火焰半径下限 (mm)
13     MAX_RADIUS_MM = 25                           # 有效火焰半径上限 (mm)
14
15     # 5. 帧率 (对于图片文件夹输入必需)
16     FPS = 20000                                   # 图片序列的帧率
17
18     # 6. 置信度阈值
19     CONFIDENCE_THRESHOLD = 0.05                   # 分割置信度阈值
20
21     # 7. 是否显示处理过程
22     SHOW_PROCESSING = False                       # 是否实时显示处理过程
23
24     # 8. 输入类型 ("auto"自动检测, "video"视频文件, "images"图片文件夹)
25     INPUT_TYPE = "auto"                          # "auto"、"video" 或 "images"
26
27     # 9. 轮廓选择比例
28     CONTOUR_RATIO = 0.6                          # 轮廓点选择比例
29
30     # 10. 间隔处理参数 (每deal_gap帧处理1帧)
31     DEAL_GAP = 5                                  # 抽帧间隔 (每 5 帧处理 1 帧)
32
33     # 11. 新增卡尔曼滤波参数
34     KALMAN_ITERATIONS = 1                        # 卡尔曼滤波迭代次数 (0表示不使用)
35
36     # 12. 数据清洗参数
37     ...(省略)
38
39     # 13. 视窗检测方式选择
40     WINDOW_DETECTION_METHOD = "manual"           # 视窗检测方式: yolo / traditional / manual
41     MANUAL_PIXEL_TO_MM = (手动计算)             # 手动比例尺 (mm/像素)
42     TRADITIONAL_KERNEL_SIZE = 18 # 传统方法的开运算核大小

```

修改后直接在 PyCharm 中右键脚本选择 `Run 'predict_lfs_calculator'` 或 `Shift+F10` 即可运行。

4.3.2 方式二：通过 PyCharm 的运行配置传递命令行参数（更灵活）

1. 在 PyCharm 顶部菜单栏点击 `Run → Edit Configurations...`。
2. 找到或新建一个 Python 运行配置，`Script path` 设为 `predict_lfs_calculator.py`。
3. 在 `Parameters` 字段中填入所需参数，例如：

```

1      --model weights/best.pt \
2      --source ./datasets/cine2tiff/test \
3      --output ./results \
4      --min-radius 8 --max-radius 25 \
5      --fps 20000 --conf 0.05 \
6      --window-method manual --manual-pixel-to-mm (手动计算) \
7      --deal-gap 5 --contour-ratio 0.6

```

常用命令行参数说明：

参数	说明	默认值
<code>--model</code>	模型权重路径	必填
<code>--source</code>	输入文件夹或视频文件	必填
<code>--output</code>	输出目录	<code>./integrated_results</code>
<code>--min-radius</code>	下临界半径 (mm)	8
<code>--max-radius</code>	上临界半径 (mm)	25
<code>--fps</code>	图片序列帧率	20000
<code>--conf</code>	分割置信度阈值	0.06
<code>--show-processing</code>	实时显示处理过程（无值则为 True）	False
<code>--input-type</code>	输入类型：auto / video / images	auto
<code>--contour-ratio</code>	用于圆拟合的轮廓点选取比例	0.6
<code>--deal-gap</code>	抽帧间隔（每 N 帧处理 1 帧）	5
<code>--window-method</code>	视窗检测方式：yolo / traditional / manual	yolo
<code>--manual-pixel-to-mm</code>	手动比例尺（仅 manual 模式有效）	0.09849
<code>--kalman-iterations</code>	卡尔曼滤波迭代次数	1
<code>--loess-frac-rt</code>	r-t 的 Loess 平滑因子	0.45
<code>--loess-frac-sbt</code>	Sb-t 的 Loess 平滑因子	0.25
<code>--window-frac</code>	数据清洗的窗口比例	0.0011
<code>--threshold-factor</code>	数据清洗的阈值因子	3.0

提示：对于高分辨率纹影图像，建议首先运行几帧以确定合适的 `MIN_RADIUS_MM` 和 `MAX_RADIUS_MM`。若不启用视窗检测（`manual` 模式），务必提供准确的 `--manual-pixel-to-mm`。

4.4 运行预测

1. 确认 Python 解释器已选择为 `YOLO` 环境（PyCharm 右下角状态栏可切换）。
2. 若使用参数区域直接修改的方式，右键编辑区选择 `Run 'predict_lfs_calculator'`；若已配置运行配置，点击运行按钮。
3. 脚本将依次执行：
 - 读取输入帧，进行模型推理；
 - 视窗检测及比例尺计算（仅第一帧）；
 - 逐帧提取火焰半径，过滤无效值，数据清洗与平滑；
 - 计算传播速度、拉伸率，三种模型拟合；
 - 显示四象限结果图并保存。

4. 处理结束后，弹出 Matplotlib 结果图窗口，关闭窗口后将在输出目录生成分析文件。

4.5 4.5 输出文件说明

所有输出文件保存在 `--output` 指定的目录下。若想快捷查看结果，则进入 `./integrated_results/` 文件夹快捷查看即可。若需要使用数据进行绘图，本程序提供了完整的处理数据，存放在 `./curve4origin/` 中。供用户自行选择。各个文件夹中的文件主要包括：

integrated_results 文件	内容
<code>./integrated_results/flame_analysis_results.csv</code>	逐帧的帧号、时间、火焰半径、面积、传播速度、拉伸率
<code>./integrated_results/flame_analysis_summary.txt</code>	三种模型的拟合结果 <code>Sb0</code> 、 <code>Lb</code> 及运行参数摘要
<code>./integrated_results/flame_analysis_results.png</code>	四象限分析图（半径-时间、速度-半径、速度-时间、速度-拉伸率）

curve4origin 文件	内容
<code>./curve4origin/LFS_data*_exp.csv</code>	用于 Origin 绘图的实验数据点（拉伸率、半径、速度）
<code>./curve4origin/LFS_data*_fit.csv</code>	三条拟合线的完整数据点（线性、Frankel、Kelly）
<code>./curve4origin/LFS_data*_params.csv</code>	无拉伸层流火焰速度 <code>Sb0</code> 、马克斯坦长度 <code>Lb</code> 结果

4.6 交互式结果回放及结果显示

若在配置中设置 `SHOW_PROCESSING = True` 或添加 `--show-processing`，处理过程中会实时显示每一帧的标注状态；处理完成后，还会进入交互式回放模式：

- **左右箭头键**：前进/后退一帧；
- **空格键**：暂停/继续播放；
- **H 键**：切换是否显示标注信息（原图/标注图）；
- **ESC 键**：退出回放。

这有助于直观检查模型分割质量与拟合效果，便于参数调优。

Important

数据处理完成后，会自动弹出处理过程动画窗口，动画将以循环方式自动播放。

首次自动播放结束后，系统会自动弹出独立的四象限分析图（Matplotlib 窗口），两者不会重叠或融合。

按 **ESC 键** 可退出动画窗口，并自动弹出四象限分析图。

若希望再次观看动画，需重新运行预测脚本。

BUG

正常情况下，动画窗口和分析图窗口不会重叠或融合。但有时会出现 [四象限分析图与动画窗口融合显示](#) 的情况，此时只需要按下 **空格键** 便可继续播放动画，直至用户按下 **ESC键**，动画窗口才会被关闭，并自动弹出四象限分析图。

5. 附录：常见问题

1. **标注快捷键冲突**：labelme 默认多边形标注快捷键为 **Ctrl+N**，矩形框为 **Ctrl+R**，请勿混淆。
2. **显存不足**：降低 **batch**、**imgsz**，或使用 **minimal** 配置，亦可关闭 **amp** 以略微降低显存占用。
3. **训练过程中开启 AMP 导致报错**：

报错信息如下：

```
1 RuntimeError: PytorchStreamReader failed reading zip archive: failed finding central directory
```

这个错误是在训练过程中的 **AMP 自动混合精度检查** 阶段发生的。Ultralytics 在开启 AMP 时，会强制加载一个官方模型 **yolo11n.pt** 来验证混合精度在您的设备上是否工作正常，但找不到有效的中央目录，导致无法读取，通常是因为：

- 文件下载不完整（网络中断）。
- 文件被错误覆盖（比如用文本形式打开后保存）。
- 文件根本不是有效的 PyTorch 权重文件。

Ultralytics 的 AMP 检查是“白盒”测试，它固定使用 **yolo11n.pt** 作为标准模型，而不是您自定义的 **YOLOv8n-seg-sphflame-v1.0.pt**。即使您用的是自己的分割模型，该检查依然会执行，这是框架的内部行为。

若开启混合精度时提示 **yolo11n.pt** 损坏，请根据以下解决方案进行操作：

方案一：

请从 [官方仓库](#) 下载完整文件，放入工作目录或脚本同级的 **./weights/** 文件夹中（因此切勿删除 **./weights/** 文件夹中的 **yolo11n.pt** 文件）。

方案二：

在您的训练参数中将 **amp** 设为 **False**，这样整个 AMP 检查流程就会被跳过，不会再尝试加载 **yolo11n.pt**。

修改 **train_args** 字典：

```
1 'amp': False, # default = True
```

这样做的代价是训练不使用混合精度，速度会稍慢，但可以避免该错误，且对于某些配置（尤其是 batch 很小的时候）影响不大。如果显存紧张，关闭 AMP 可能更稳定。